附录A：术语解释

> **使用说明**：本附录收录全书核心术语,按字母顺序排列。每个术语包含中文对照、定义、首次定义章节和典型使用场景,方便读者快速查阅。

A.1 核心术语表

Agent（智能代理）

中文对照：智能代理 / 智能体

定义：一种能够自主感知环境、做出决策并执行行动以达成目标的软件系统。与传统的Chatbot(聊天机器人)不同,Agent是**目标驱动式**的:给它一个目标,它会自主规划步骤、调用工具、处理中间结果和失败,直到达成目标或判断无法完成。

业界也有一个更简洁的工程视角定义（来自 Simon Willison， `datasette` 作者）：

> **"智能体在循环中运行工具以实现目标。"**（Agents run tools in a loop to achieve a goal.）

这个定义抓住了三个核心要素：**循环**（反复迭代直到完成）、**工具**（具备与外部世界交互的能力）、**目标**（不是单轮问答，而是面向结果）。

首次定义章节：第00章 前言

核心特征：

- 自主性：无需人工干预即可执行任务
- 目标驱动：围绕明确目标展开行动
- 工具调用：具备与外部世界交互的能力
- 状态管理：跨多轮对话和工具调用维护上下文

典型使用场景：

- 自动化客户服务系统
- 数据分析与报告生成
- 代码辅助开发

- 复杂业务流程自动化

相关术语：Chatbot、LLM、Function Calling、Skill

API（Application Programming Interface，应用程序编程接口）

中文对照：应用程序编程接口

定义：软件系统之间进行通信的约定和接口。在Agent开发中,API主要指LLM供应商提供的调用接口(如OpenAI API、Claude API等),用于发送Prompt并接收模型响应。

首次定义章节：第01章 AI平台和api-key管理

核心概念：

- **API Key**：身份认证凭证,用于标识调用者身份和计费
- **Rate Limit**：调用频率限制,防止滥用
- **Endpoint**：API的服务地址

典型使用场景：

- 调用OpenAI GPT-4进行对话
- 调用Embedding API生成向量表示
- 调用Whisper API进行语音识别

注意事项：

- API Key需安全存储,禁止硬编码在代码中
- 需要处理Rate Limit导致的调用失败
- 建议实现熔断机制防止雪崩效应

相关术语：LLM、Context、Token

Chunk（文本分块）

中文对照：文本分块 / 文本切片

定义：将长文档切分为较小片段的过程,是RAG系统的核心预处理步骤。Chunk的质量直接影响检索效果和生成质量。

首次定义章节：第08章 RAG检索增强生成

核心策略：

- **固定长度切分**：按字符数或Token数切分,简单但可能破坏语义
- **语义边界切分**：按段落、句子边界切分,保留语义完整性
- **Parent-Child切分**：小块用于检索,大块用于返回上下文
- **AST切分**：针对代码按语法结构(函数、类)切分

典型参数：

- `chunk_size` ：分块大小(如256、512、1024 tokens)
- `chunk_overlap` ：分块重叠区域,避免信息丢失

使用场景：

- 构建知识库索引
- 文档问答系统
- 代码仓库分析

相关术语：RAG、Embedding、Vector、Token

Context（上下文）

中文对照：上下文

定义：LLM在一次调用中接收的所有输入信息,包括系统指令、对话历史、检索到的知识库内容和当前用户问题。Context是Agent的"工作记忆",决定了模型对当前任务的理解深度。

首次定义章节：第02章 核心交互_Context和Prompt管理

四大核心要素：

要素	层级	作用	变化频率
System Message	宪法层	定义角色、能力边界、输出格式	极低
RAG Context	知识层	提供外部知识库检索结果	高
History Message	记忆层	维持对话连贯性	中

要素	层级	作用	变化频率
User Message	触发层	用户当前输入	极高

上下文窗口：LLM对Context的最大Token限制(如GPT-4 Turbo为128k tokens)。超出限制需要采用压缩或截断策略。

管理策略：

- **滑动窗口**：只保留最近N轮对话
- **摘要压缩**：用LLM总结旧对话
- **动态回溯**：根据相关性动态召回历史信息

相关术语：Prompt、Token、Session Memory、RAG

Embedding（嵌入向量）

中文对照：嵌入向量 / 向量嵌入

定义：将文本、图像等非结构化数据转换为高维稠密向量表示的过程。Embedding向量能捕获语义相似性,使计算机能够理解"苹果"和"水果"在语义上相近。

首次定义章节：第08章 RAG检索增强生成

核心特性：

- **维度**：向量的长度(如768、1536、3072维)
- **语义距离**：向量间的余弦相似度反映语义相似性
- **多语言支持**：优秀模型支持中英文跨语言检索

典型模型：

- OpenAI text-embedding-3-large
- BAAI/bge-m3(开源,支持中英文)
- Cohere embed-v3

使用场景：

- RAG检索：将问题和文档转为向量进行相似度匹配

- 聚类分析：将相似文本聚集在一起
- 推荐系统：基于向量相似度推荐相关内容

代码示例：

```python
from openai import OpenAI
client = OpenAI()

response = client.embeddings.create(
    model="text-embedding-3-large",
    input="苹果是一种水果"
)
vector = response.data[0].embedding  # 输出: [0.123, -0.456, ...]
```

相关术语：Vector、RAG、Chunk、Token

Function Calling（函数调用）

中文对照：函数调用 / 工具调用

定义：LLM生成结构化JSON以调用外部工具的能力。Function Calling是Agent从"说"到"做"的桥梁,使AI能够查询实时数据、操作数据库、发送邮件等。

首次定义章节：第04章 能力扩展_FunctionCalling

核心原理：

- **决策与执行分离：**LLM只生成"调用意图",不执行代码
- **标准化定义：**通过JSON Schema描述工具的名称、参数和用途
- **闭环反馈：**执行结果返回给LLM,用于生成最终回复

生命周期流程：

```
 用户提问 → LLM决策 → 生成Tool Call JSON →
本地执行 → 返回结果 → LLM生成最终回复
```

工具定义示例：

```json
{
  "name": "get_weather",
  "description": "查询指定城市的实时天气",
  "parameters": {
    "type": "object",
    "properties": {
```

```
        "city": {
          "type": "string",
          "description": "城市名称,如'北京'"
        }
      },
      "required": ["city"]
    }
  }
```

安全注意事项：

- 禁止执行任何数据删除操作
- 需要实现参数校验防止幻觉
- 敏感操作需要人工审核

相关术语：Agent、Skill、Tool Executor、Context

LLM（Large Language Model，大语言模型）

中文对照：大语言模型

定义：基于Transformer架构训练的超大规模神经网络,能够理解和生成自然语言文本。LLM是Agent的"大脑",负责推理、规划和生成。

首次定义章节：第00章 前言

核心能力：

- **自然语言理解：**理解用户意图和上下文
- **文本生成：**生成流畅、连贯的回复
- **推理能力：**进行逻辑推理和问题求解
- **工具调用：**生成结构化的Function Call

主流模型：

- OpenAI GPT-4 / GPT-4 Turbo
- Anthropic Claude 3.5 Sonnet
- Google Gemini Pro
- Meta Llama 3

关键概念：

- **Token**：文本的最小处理单位
- **Context Window**：单次调用的最大Token数
- **Temperature**：控制输出随机性(0-2)

使用场景：

- 对话系统
- 文本生成与摘要
- 代码生成与解释
- 知识问答

相关术语：Agent、Token、Context、API

Memory（记忆）

中文对照：记忆

定义：Agent存储和召回历史信息的能力。根据生命周期和作用范围,分为短期记忆(Session Memory)和长期记忆(Long-term Memory)。

首次定义章节：第06章 短期记忆_SessionMemory

分类：

类型	生命周期	作用	实现方式
Session Memory	会话级	维持对话连贯性	Redis、内存数据库
Long-term Memory	永久	跨会话知识保留	向量数据库、关系数据库

Session Memory核心设计：

- 基于角色的消息结构(user/assistant/tool)
- Tool Call配对完整性
- 上下文窗口管理策略

长期记忆核心设计：

- 用户隔离
- 数据生命周期管理
- 权限控制

相关术语：Context、Token、Session、RAG、Vector

Pipeline（流程编排）

中文对照：流程编排 / 管道

定义：将多个Agent处理节点按特定逻辑组合成完整工作流的技术。Pipeline是Agent系统的"神经系统",协调各组件有序工作。

首次定义章节：第11章 流程编排_Pipeline

三大基础模式：

模式	说明	生活类比
顺序链	节点依次执行	餐厅后厨:洗菜→切菜→烹饪→装盘
条件分支	根据条件选择路径	医院分诊:判断病情→分流科室
循环迭代	重复执行直到满足条件	写作:初稿→审阅→修改→定稿

高级架构：

- **DAG(有向无环图)**：处理复杂依赖关系
- **并行执行：**无依赖节点同时执行
- **人机协作节点：**关键环节插入人工审核

核心价值：

- 可观测性：清晰追踪执行链路
- 可控性：在关键节点插入审核
- 鲁棒性：重试和降级机制

代码示例：

```python
from langchain.schema.runnable import RunnableParallel

# 并行执行两个独立任务
pipeline = RunnableParallel({
    "news": fetch_news_chain,
    "stocks": fetch_stock_chain
})
result = pipeline.invoke({"topic": "AI"})
```

相关术语：Agent、State、ReAct、Plan-and-Execute

Planner（规划器）

中文对照：规划器

定义：负责将复杂目标拆解为可执行子任务的组件。Planner不执行具体操作,只输出任务列表。

首次定义章节：第09章 规划与推理

核心职责：

- 任务分解：将大目标拆解为原子任务
- 依赖识别：识别任务间的前置依赖关系
- 顺序安排：确定任务的执行顺序

工作流程：

```
 用户目标 → Planner分析 → 输出任务列表 →
Executor执行 → 监控进度 → 动态重规划(可选)
```

规划粒度控制：

- **过粗**："赚钱"(无法执行)
- **适中**："收集数据→分析数据→生成报告"
- **过细**："打开浏览器→点击按钮→输入文字"(脆弱)

最佳实践：

- 采用分层规划:高层规划宏观步骤,底层Worker执行细节
- 设置重规划触发条件:执行失败或环境变化

- 引入Self-Reflection机制审查规划质量

相关术语：ReAct、Executor、Sub-Agent、Self-Reflection

Prompt（提示词）

中文对照：提示词

定义：发送给LLM的指令文本,用于引导模型生成符合预期的输出。Prompt是人与AI沟通的主要方式,其设计质量直接影响输出质量。

首次定义章节：第02章 核心交互_Context和Prompt管理

核心类型：

类型	作用	示例
System Prompt	定义角色、能力边界	"你是一名专业的Python开发工程师"
Task Prompt	描述具体任务	"请为以下函数编写单元测试"
Few-shot Prompt	提供示例引导输出	"示例1:输入→输出\n示例2:..."

工程化实践：

- **版本控制**：像管理代码一样管理Prompt
- **A/B测试**：对比不同Prompt效果
- **变量注入**：使用模板引擎动态填充

代码示例：

```
python
from jinja2 import Template

template = Template("""
你是一名{{role}},专注于{{domain}}。

当前任务:{{task}}
输出格式:{{format}}
""")

prompt = template.render(
    role="数据分析师",
    domain="用户行为分析",
    task="分析用户留存率下降原因",
    format="JSON格式报告"
)
```

相关术语：Context、LLM、Token、System Message

RAG（Retrieval-Augmented Generation，检索增强生成）

中文对照：检索增强生成

定义：结合外部知识库检索和LLM生成的技术架构。RAG通过"先检索相关文档,再让LLM基于文档回答"的方式,解决LLM知识滞后和幻觉问题。

首次定义章节：第08章 RAG检索增强生成

核心流程：

```
用户问题 → Embedding向量化 → 向量检索相似文档 →
文档作为Context注入Prompt → LLM基于文档生成回答
```

四大应用场景：

场景	难度	核心挑战
智能客服/FAQ	⭐	口语化匹配
企业知识库	⭐⭐	表格切片、权限隔离
法律合同审查	⭐⭐⭐	零幻觉、精准引用
代码仓库分析	⭐⭐⭐⭐	跨文件依赖、AST解析

进阶技术：

- **Parent-Child切片**：小块检索、大块返回
- **混合检索+Rerank**：向量+关键词融合
- **AST切片**：按代码语法结构切分

评估指标：

- **召回率**：检索到的相关文档占所有相关文档的比例
- **准确率**：检索到的文档中真正相关的比例
- **答案准确性**：生成的答案是否正确

相关术语：Embedding、Vector、Chunk、Context、LLM

ReAct（Reason + Act）

中文对照：推理-行动循环框架

定义：一种经典的Agent规划架构,将"推理"(Reason)和"行动"(Act)交织形成闭环。ReAct通过Thought→Action→Observation三步循环,逐步推进任务完成。

首次定义章节：第09章 规划与推理

核心循环：

```
 Thought(思考): 分析当前状态,规划下一步
 ↓
Action(行动): 选择工具并生成调用参数
 ↓
Observation(观察): 获取工具执行结果
 ↓
(判断是否完成) → 未完成则继续循环
```

实战案例：

```
 用户: 苹果公司最新股价是多少?买入100股需要多少钱?

[第1轮]
Thought: 需要查询苹果股价,我还没有这个信息
Action: search_stock_ticker(symbol="AAPL")

[第2轮]
Observation: 当前股价178.50美元
Thought: 已获取股价,现在计算总成本
Action: calculator(expression="178.50 * 100")

[第3轮]
Observation: 17850.0
Thought: 已获取所有信息,可以回答了
Final Answer: 苹果公司最新股价是178.50美元,买入100股需要17,850美元
```

优势：

- 灵活性强,适合动态场景
- 可解释性好,推理过程透明

局限：

- 缺乏全局观,容易在中途偏离目标
- Token消耗大,每次循环都需重新加载历史
- 容易陷入死循环

相关术语：Planner、Pipeline、Self-Reflection、Function Calling

Self-Reflection（自我反思）

中文对照：自我反思

定义：Agent在执行动作后,通过"批评家"角色审查自己的输出,并根据反馈进行修正的能力。Self-Reflection是Agent实现自我进化的关键机制。

首次定义章节：第09章 规划与推理

核心流程：

```
Draft(初稿) → Critic(批评审查) → Refine(修改完善)
```

应用场景：

- **代码生成：**生成代码→检查语法→修正Bug→再次验证
- **SQL生成：**生成SQL→检查语法和性能→优化查询→执行
- **报告写作：**生成初稿→审阅结构和逻辑→修改润色→定稿

Prompt示例：

```
[Draft]
生成一段Python代码读取CSV文件并计算平均值

[Critic Prompt]
请以第三视角审查上述代码:

1. 是否处理了文件不存在的异常?

2. 是否处理了数据类型转换异常?

3. 性能是否有优化空间?

[Refine Prompt]
根据批评意见修改代码,确保健壮性和性能
```

注意事项：

- 设置最大反思次数,避免无限循环
- 批评Prompt需要明确审查维度
- 需要有明确的"通过标准"

相关术语：ReAct、Planner、Self-Evolution

Session Memory（短期记忆）

中文对照：短期记忆 / 会话记忆

定义：Agent在单次会话期间的临时记忆存储,用于维持对话连贯性。Session Memory类似于计算机的RAM,会话结束后自动清空。

首次定义章节：第06章 短期记忆_SessionMemory

核心特性：

- **生命周期：**会话级,关闭窗口或超时后清空
- **作用：**解决"前言不搭后语"问题
- **隔离性：**不同用户的Session互不干扰

消息结构设计：

```
json
{
  "role": "user|assistant|tool|system",
  "content": "消息内容",
  "tool_calls": [{"id": "call_123", "name": "get_weather", "args": {...}}],
  "tool_call_id": "call_123"
}
```

上下文窗口管理策略：

- **滑动窗口：**只保留最近N轮
- **摘要压缩：**用LLM总结旧对话
- **混合策略：**System Message + 摘要 + 最近对话

存储方案：

- **Redis：**高性能,适合大规模并发
- **内存字典：**简单,适合小规模应用

代码示例：

```
python
import redis

class SessionMemory:
    def __init__(self, redis_client):
        self.redis = redis_client

    def get_messages(self, session_id: str):
```

```
        return self.redis.get(f"session:{session_id}")

    def add_message(self, session_id: str, message: dict):
        messages = self.get_messages(session_id) or []
        messages.append(message)
        self.redis.set(f"session:{session_id}", messages)
```

相关术语：Memory、Context、Token、Session

Skill（技能）

中文对照：技能

定义：比Function更高维度的抽象,是Agent的业务解决方案。一个Skill可能包含特定的Prompt策略、一系列有序的工具调用、专用知识库资源和状态管理。

首次定义章节：第05章 技能体系_AgentSkills定义和组织

Skill vs Function：

维度	Function(工具)	Skill(技能)
层级	底层原子操作	上层业务封装
示例	read_file、http_get	analyze_sales_report
复用性	通用性强	领域性强
逻辑	无状态执行	包含推理和流程控制
生命周期	瞬时执行	可能包含多轮交互

结构化定义：

```
json
{
  "metadata": {
    "name": "financial_report_generator",
    "display_name": "财务报表生成器",
    "tags": ["finance", "report"],
    "category": "financial-analysis"
  },
  "input_schema": {...},
  "output_schema": {...},
  "tools required": ["sql query", "excel generator"],
  "knowledge bases": ["kb financial_regulations"],
  "system_prompt_template": "..."
}
```

技能管理：

- **集中式注册表**：统一管理所有技能定义
- **动态发现**：运行时扫描加载新技能
- **语义匹配**：根据任务描述自动匹配最相关技能

相关术语：Function Calling、Agent、Prompt、Knowledge Base

SubAgent（子代理）

中文对照：子代理 / 子智能体

定义：在多智能体系统中,负责执行特定子任务的独立Agent实例。SubAgent通常由主Agent调度和管理,完成特定领域或特定步骤的工作。

首次定义章节：第09章 规划与推理

核心特性：

- **独立性**：拥有自己的Prompt、工具和记忆
- **专业性**：专注于特定任务领域
- **协作性**：通过消息传递与主Agent或其他SubAgent通信

应用场景：

- **代码审查Agent**：专门负责代码质量检查
- **数据分析Agent**：专门负责数据处理和可视化
- **测试Agent**：专门负责生成和执行测试用例

调度模式：

- **顺序执行**：SubAgent A完成后,SubAgent B开始
- **并行执行**：多个SubAgent同时工作
- **层级调用**：主Agent调度SubAgent,SubAgent再调度Worker Agent

注意事项：

- 需要监控SubAgent执行状态

- 设置超时机制防止阻塞
- 实现结果聚合和错误处理

相关术语：Agent、Planner、Multi-Agent、Pipeline

Token（词元）

中文对照：词元 / 标记

定义：LLM处理文本的最小单位。一个Token可以是一个单词、一个汉字或一个字符片段。Token数量直接影响API调用成本和上下文窗口占用。

首次定义章节：第02章 核心交互_Context和Prompt管理

核心概念：

- **Tokenization**：将文本切分为Token的过程
- **Token计费**：LLM API按Token数量收费
- **Context Window**：LLM单次调用的最大Token数

Token vs 字符数估算：

- **英文**：1 Token ≈ 4个字符 ≈ 0.75个单词
- **中文**：1 Token ≈ 1.5个汉字
- **代码**：1 Token ≈ 2-3个字符

Token计数工具：

```
python
import tiktoken

encoder = tiktoken.encoding_for_model("gpt-4")
text = "你好,世界!"
token_count = len(encoder.encode(text))  # 输出: 约4-5个Token
```

成本控制策略：

- 压缩Prompt,去除冗余信息
- 使用摘要压缩旧对话

- 选择合适模型(大模型贵,小模型便宜)

相关术语：Context、LLM、API、Session Memory

Vector（向量）

中文对照：向量

定义：在LLM语境中,Vector通常指Embedding生成的数值数组,用于表示文本的语义特征。向量间的距离反映文本间的语义相似性。

首次定义章节：第02章 核心交互_Context和Prompt管理

核心概念：

- **维度**：向量的长度,如768维、1536维
- **余弦相似度**：衡量两个向量方向相似性的指标
- **向量数据库**：专门存储和检索向量的数据库

相似度计算：

```
python
import numpy as np

def cosine similarity(vec1, vec2):
    """计算两个向量的余弦相似度"""
    dot product = np.dot(vec1, vec2)
    norm = np.linalg.norm(vec1) * np.linalg.norm(vec2)
    return dot_product / norm

# 相似度范围: [-1, 1]

# 1表示完全相同, 0表示正交, -1表示完全相反
```

典型向量数据库：

- **Milvus**：开源,高性能,适合生产环境
- **Pinecone**：托管服务,易上手
- **Weaviate**：支持混合检索
- **Chroma**：轻量级,适合小规模应用

使用场景：

- RAG检索：计算问题向量与文档向量的相似度
- 语义搜索：找到语义相近的文档
- 聚类分析：将相似文本归为一类

相关术语：Embedding、RAG、Chunk、Vector Database

A.2 术语使用规范

中英文混用标准

为保持全书术语一致性,建议遵循以下规范:

英文术语	中文对照	推荐用法
Agent	智能代理	首次出现用"Agent(智能代理)",后续直接用Agent
Context	上下文	首次出现用"Context(上下文)",后续视上下文选择
Prompt	提示词	直接使用Prompt
Token	词元	首次出现用"Token(词元)",后续直接用Token
Embedding	嵌入向量	直接使用Embedding
Vector	向量	技术描述用Vector,通俗解释用向量
Function Calling	函数调用	首次出现用英文+中文,后续用Function Calling
RAG	检索增强生成	首次出现用"RAG(检索增强生成)",后续用RAG

术语分类索引

核心概念类:

- Agent, LLM, API, Token

交互与记忆类:

- Context, Prompt, Memory, Session Memory

能力扩展类:

- Function Calling, Skill, SubAgent

知识管理类:

- RAG, Embedding, Vector, Chunk

规划与编排类:

- ReAct, Planner, Pipeline, Self-Reflection

A.3 快速查找表

按字母顺序索引

字母	术语
A	Agent, API
C	Chunk, Context
E	Embedding
F	Function Calling
L	LLM
M	Memory
P	Pipeline, Planner, Prompt
R	RAG, ReAct
S	Self-Reflection, Session Memory, Skill, SubAgent
T	Token
V	Vector

按章节索引

章节	核心术语
第00章 前言	Agent, LLM
第01章 AI平台管理	API, Token
第02章 Context与Prompt	Context, Prompt, Token
第04章 Function Calling	Function Calling, Agent, LLM
第05章 技能体系	Skill, Function Calling
第06章 短期记忆	Session Memory, Token, Context
第08章 RAG	RAG, Embedding, Vector, Chunk
第09章 规划与推理	ReAct, Planner, Self-Reflection, SubAgent
第11章 流程编排	Pipeline, Agent, State

A.4 参考资源

官方文档

- [OpenAI API Documentation]
- [Anthropic Claude Documentation]
- [LangChain Documentation]

学术论文

- **ReAct**: "ReAct: Synergizing Reasoning and Acting in Language Models" (Yao et al., 2022)
- **RAG**: "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks" (Lewis et al., 2020)
- **Chain-of-Thought**: "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models" (Wei et al., 2022)

开源项目

- **LangChain**: Agent开发框架
- **LlamaIndex**: RAG专用框架
- **AutoGPT**: 自主Agent实验项目
- **Milvus**: 开源向量数据库

版本信息：

- 附录版本：v1.0
- 最后更新：2026-03-20
- 收录术语：17个核心术语

使用建议：

1. 初次阅读时浏览A.1节了解核心概念
2. 阅读正文时遇到陌生术语可快速查阅
3. 通过A.3快速查找表定位术语定义章节